Task-2 on AWS_VPC

1. **Create VPC with 2 private and 2 public subnets.**

Steps:

1. Login to your AWS Management Console.

2. Search for VPC in the search bar.

3. Open VPC.

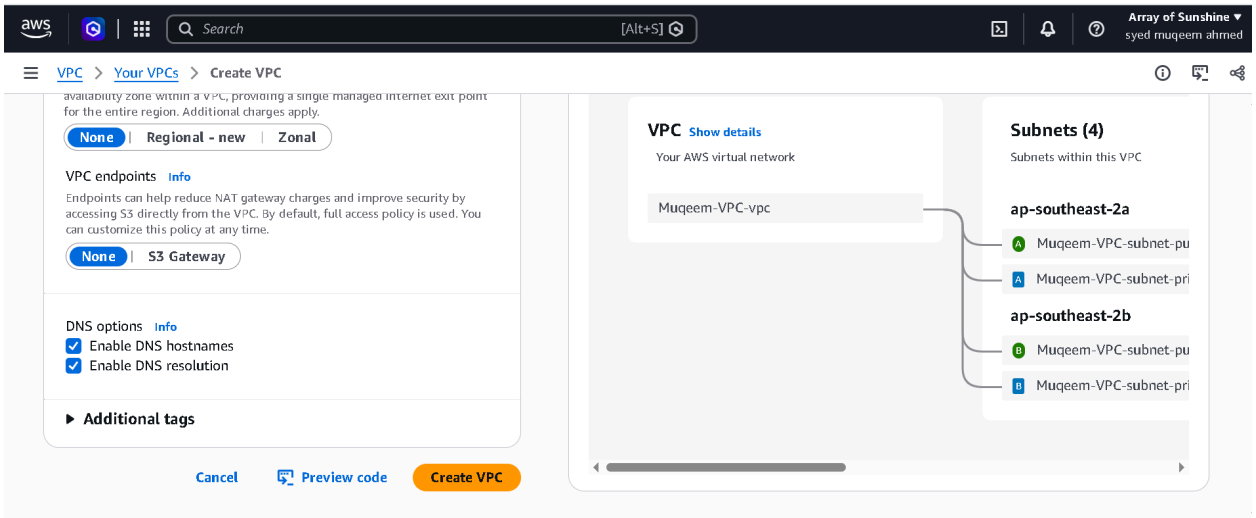
4. Click Your VPCs from the left menu.

3. Create VPC

Click Create VPC.

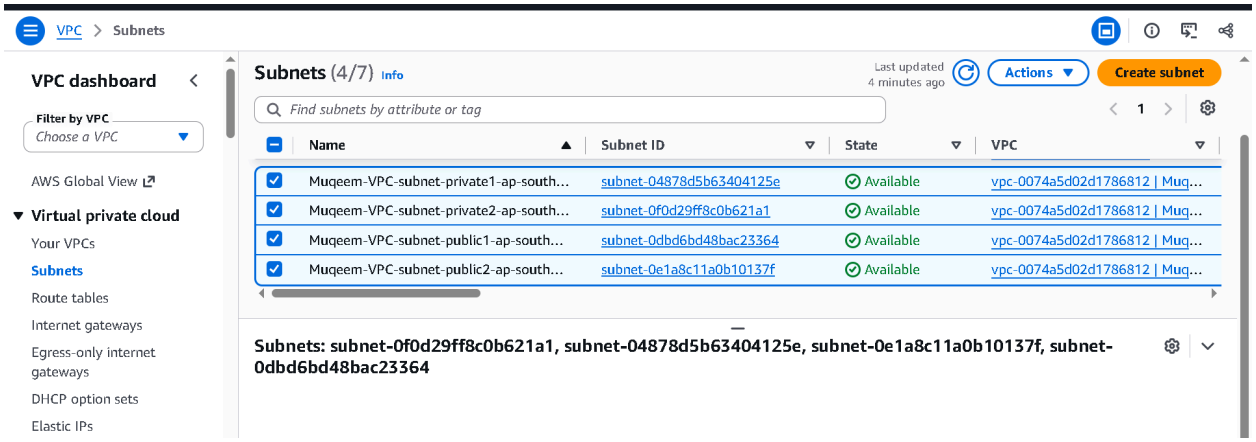
Select:----VPC and more

Enter these details:



Click create VPC

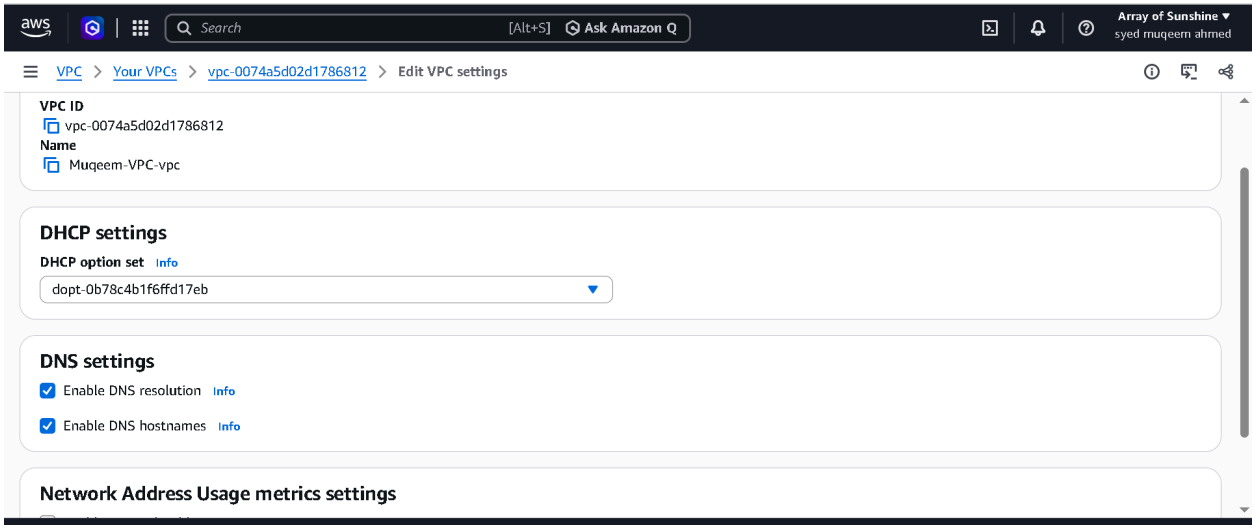
After creation, go to Your VPCs → Subnets and verify that all four subnets are created.



2. Enable DNS Hostname in VPC.

Steps:

1. Got VPC dashboard
2. Click Actions button
3. Select edit VPC settings, find DNS settings
4. Enable DNS Settings by clicking the checkboxes.



3. Enable Auto Assign Public IP in 2 public subnets.

Steps:

1. Go to your AWS VPC Dashboard.
2. From the left-side menu, click Subnets.
3. Find your two public subnets:

- Muqeem-VPC-subnet-public1
- Muqeem-VPC-subnet-public2

Step 2: Select the first public subnet

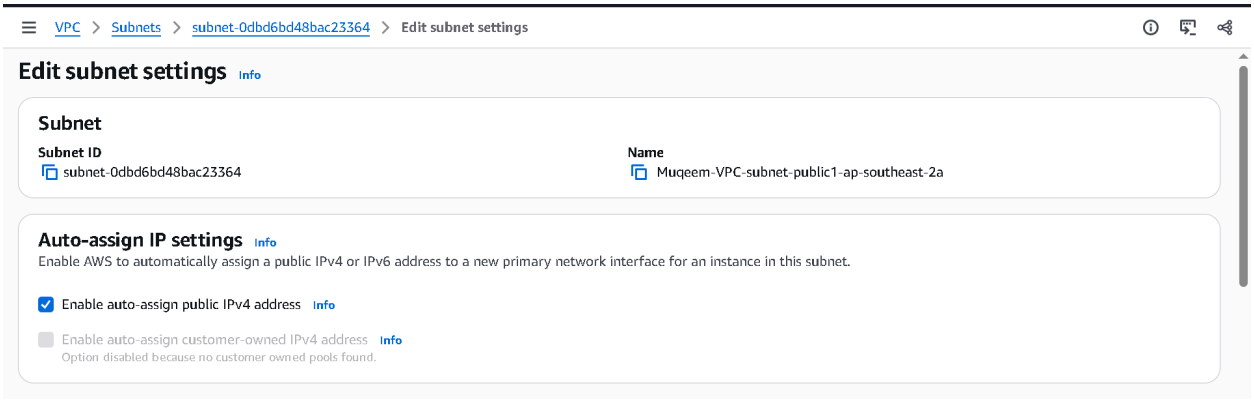
1. Select the checkbox next to your first public subnet.
2. Click Actions (top-right).
3. Find Edit subnet settings and click it.

Step 3: Enable Auto-assign Public IPv4

You'll see a setting named:

Auto-assign public IPv4 address

- Check the checkbox.
- Click Save.



4. Add 2 private subnets in private route table.

Steps:

Step 1: Open Route Tables

1. Go to AWS VPC Dashboard.
2. On the left-side menu, click Route Tables.
3. Find these two route tables:

- Muqeem-VPC-rtb-private1
- Muqeem-VPC-rtb-private2

Step 2: Associate the first private subnet

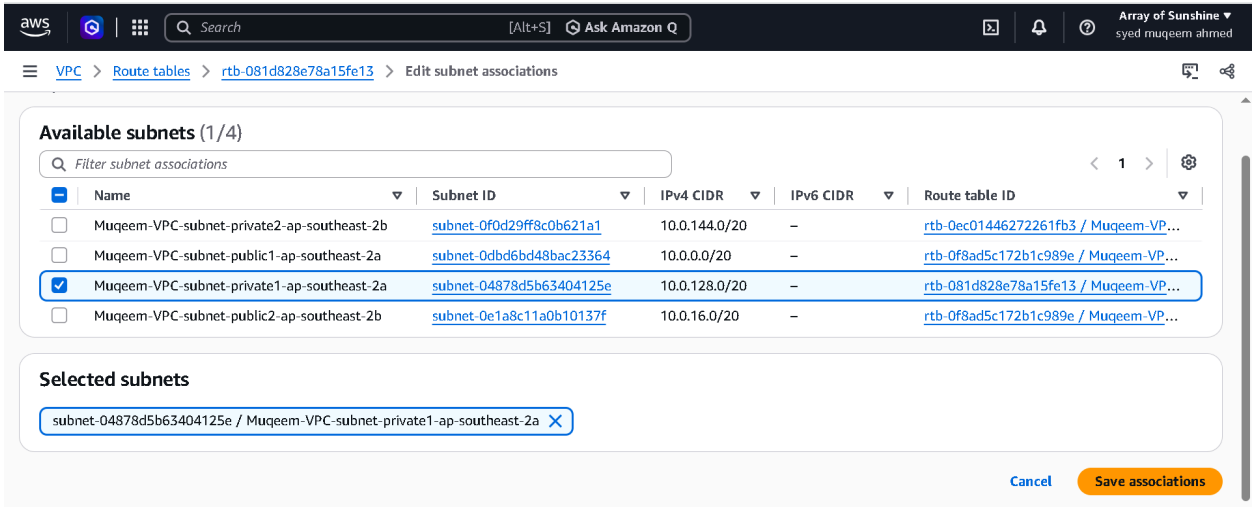
1. Select the checkbox for Muqeem-VPC-rtb-private1.
2. In the bottom panel, click Subnet associations.
3. Click Edit subnet associations.
4. You'll see available subnets.

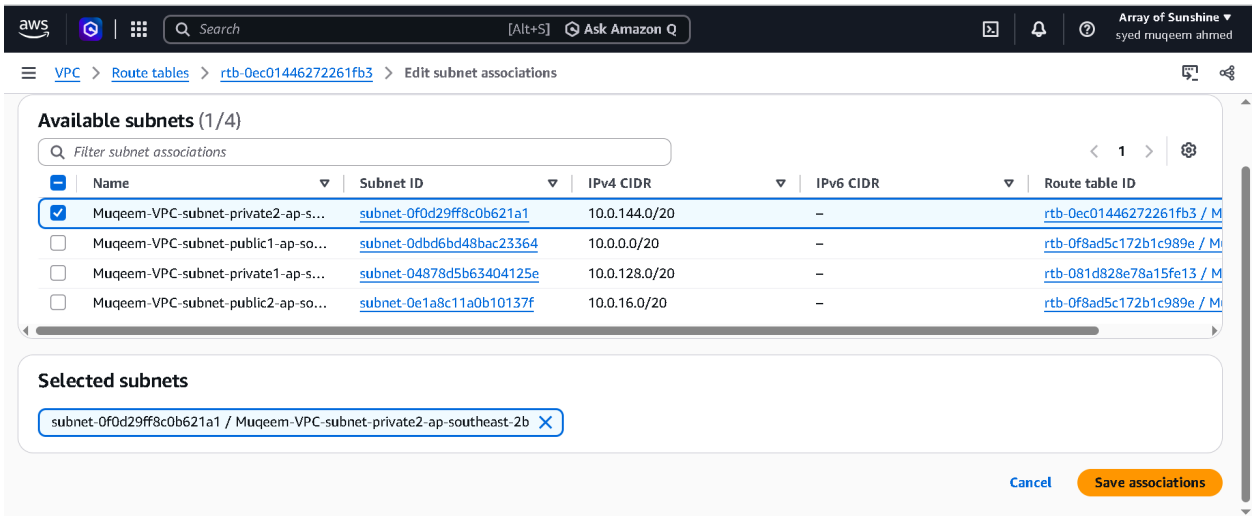
5. Select your first private subnet:

Muqeem-VPC-subnet-private1

6. Click Save associations.

Follow the same steps for Muqeem-VPC-subnet-private2





5. Add 2 public subnets in public route table.

Step 1: Open Route Tables

1. Go to AWS VPC Dashboard.
2. Click Route Tables from the left menu.
3. Find Muqeem-VPC-rtb-public.

4. Select its checkbox.

Step 2: Edit Subnet Associations

1. In the bottom panel, click Subnet associations.

2. Click Edit subnet associations.

3. You'll see a list of available subnets.

Step 3: Select both Public Subnets

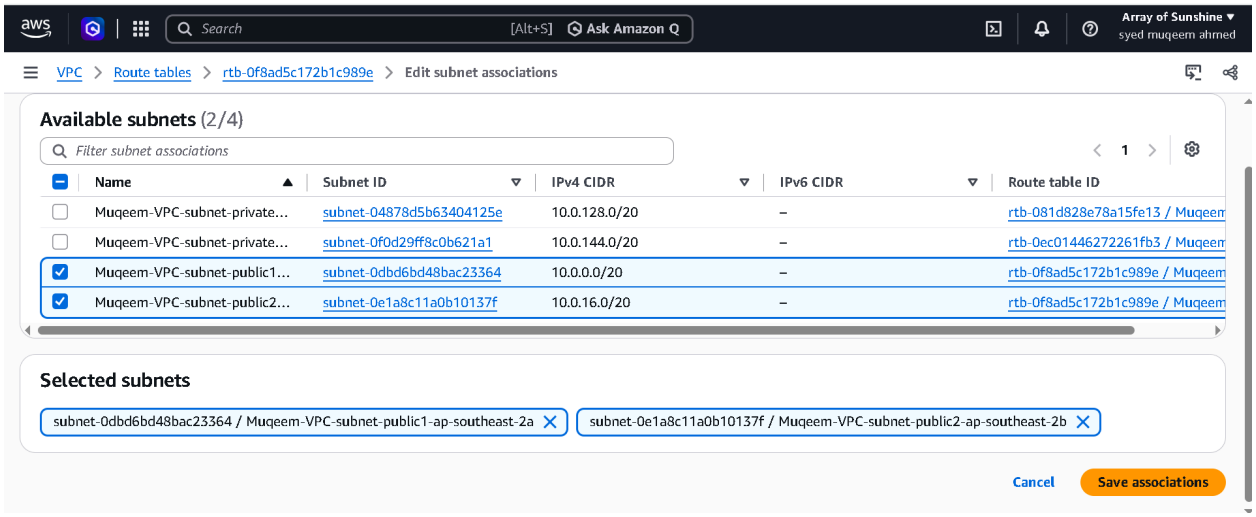
Select these two subnet checkboxes:

- Muqeem-VPC-subnet-public1

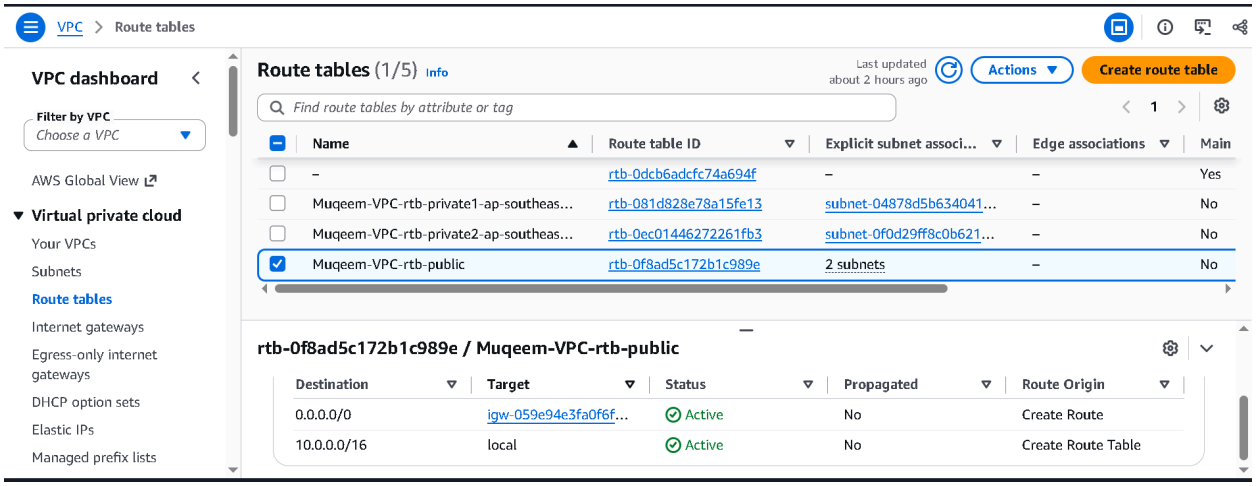
- Muqeem-VPC-subnet-public2

Make sure you select **both public subnets**, not private ones.

4. Click Save associations.



6. Public route table will have the routes to internet and local.



7. **Create EC2 in public subnet with t2.micro and install PHP.**

Part 1: Launch EC2 Instance

Step 1: Open EC2 Dashboard

1. Login to AWS Management Console.

2. Search for EC2.

3. Open the EC2 Dashboard.

4. Click Instances from the left menu.

5. Click Launch instances.

Step 2: Configure EC2 instance

Enter the following details:

Name and tags—Muqeem-PHP-Server—Amazon Linux 2023 AMI—t3.micro–Select an existing key pair

Network settings

VPC——————— Muqeem-VPC

Subnet— One of your Public Subnets

Auto-assign Public IP——— Enable

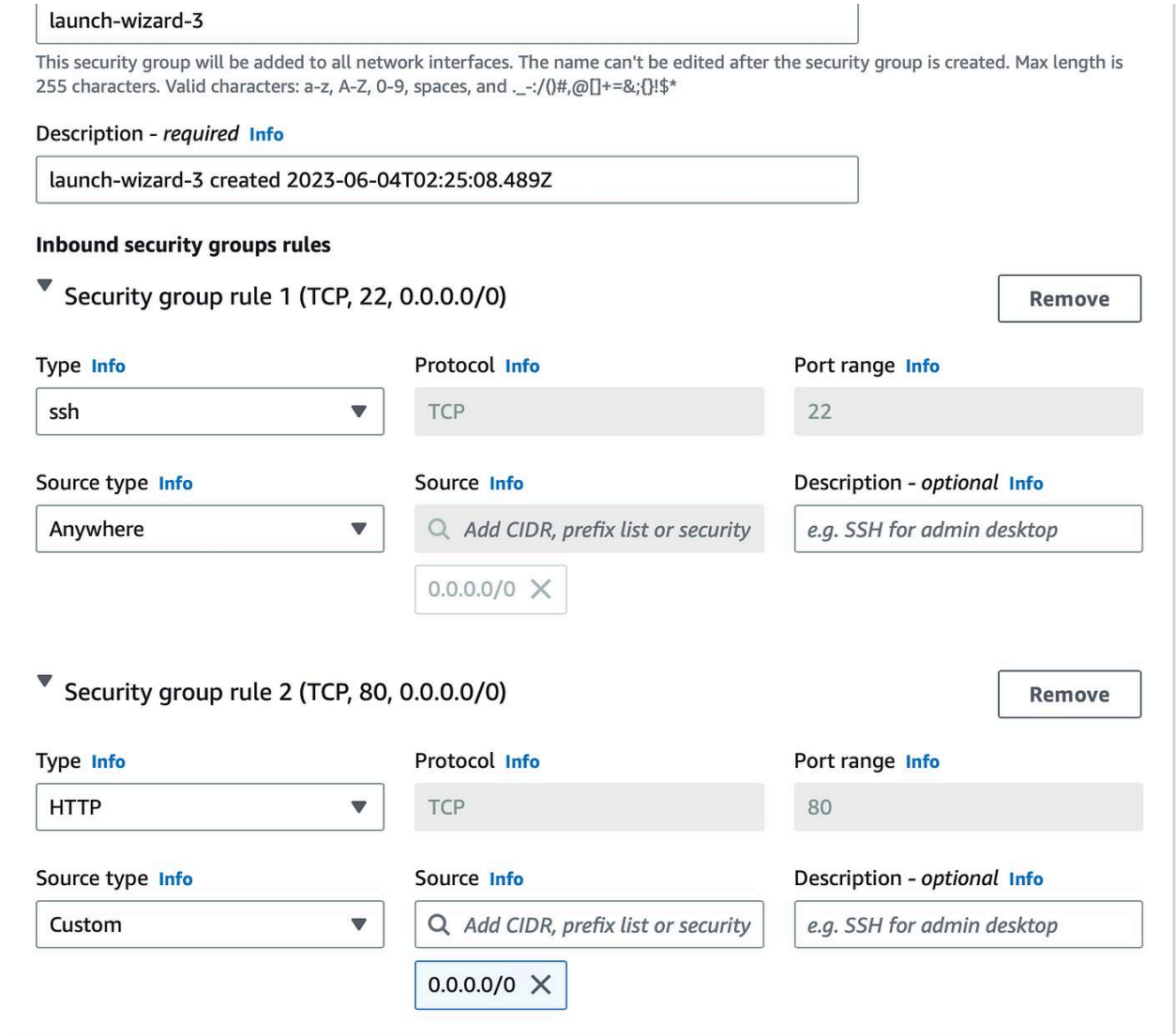
Firewall————— Create new Security Group

Step 3: Configure Security Group

A Security Group acts like a firewall for your EC2 instance.

Under Configure Security Group, add these inbound rules:

Type	Port	Source
SSH	22	My IP
HTTP	80	Anywhere (0.0.0.0/0)



Why these rules?

- SSH (22): To connect to your EC2 instance.
- HTTP (80): To access your PHP website through a browser.

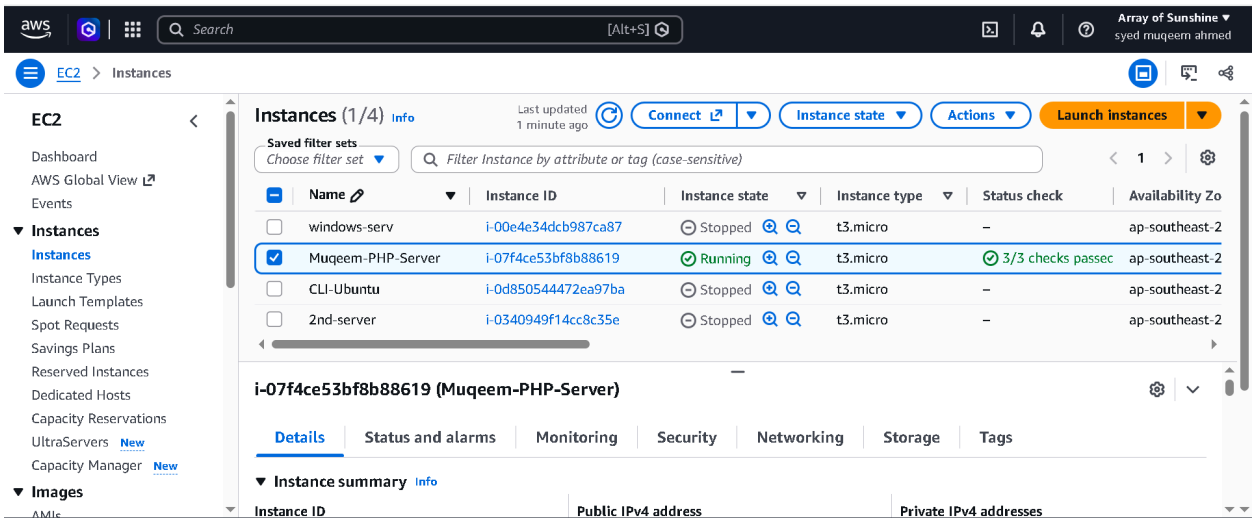
Keep the default outbound rule.

Step 4: Configure Storage

Keep the default storage settings (usually 8 GiB).

Step 5: Launch Instance

1.Review your configuration.
2.Click Launch instance.
3.Click View all instances.
4.Wait until:
- Instance state = Running
- Status checks = 2/2 checks passed



Part 2: Connect to EC2
Once your instance is running:
1. Select your instance.
2. Click Connect.
3. Select EC2 Instance Connect.
4. Click Connect.
A terminal will open in your browser.

Part 3: Install PHP

We'll use Amazon Linux 2023.

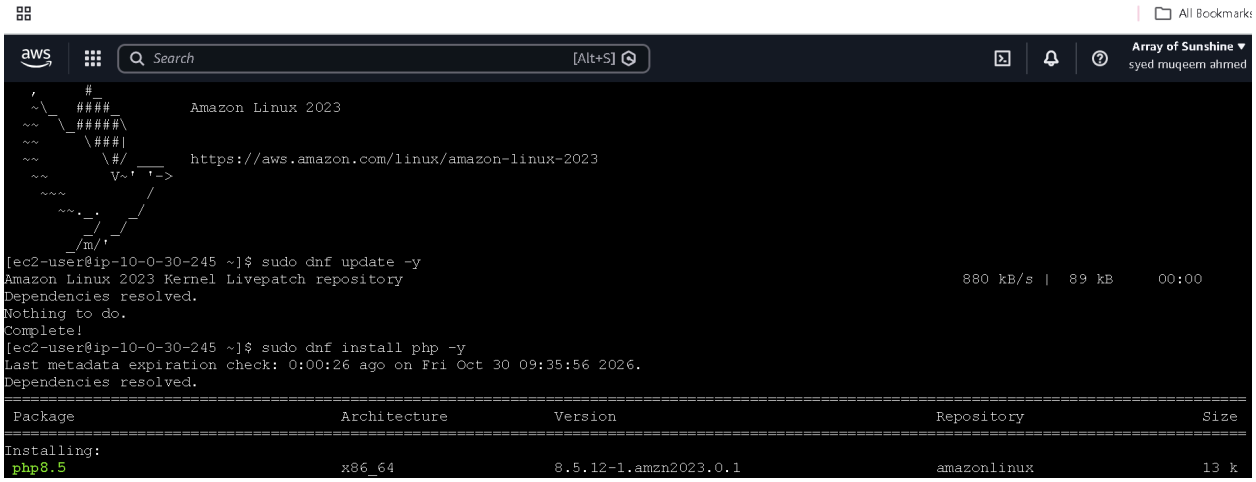
Step 1: Update packages

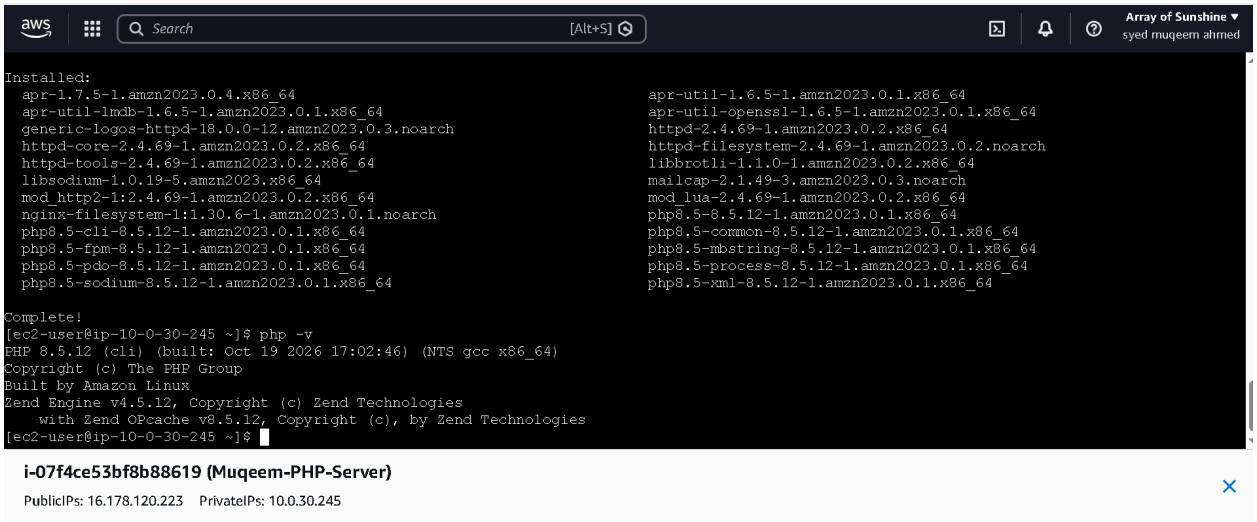
sudo dnf update -y

Step 2: Install PHP

sudo dnf install php -y

Step 3: Verify PHP installation —--- **php -v**





8. Configure NAT gateway in public subnet and connect to private instance.

Step 1: Open NAT Gateways

1. Open AWS Console.
2. Search for VPC.
3. In the left navigation menu, click NAT Gateways.
4. Click Create NAT Gateway.

Step 2: Configure NAT Gateway

Name : Muqeem-NAT-Gateway

Subnet : Your Public Subnet 1

Connectivity type : Public

Elastic IP allocation: Allocate Elastic IP

Click Create NAT Gateway.

Wait until the NAT Gateway state becomes `Available`.

3. Configure Private Route Tables

Now we need to tell AWS that private subnet traffic should go through the NAT Gateway.

Step 1: Open Route Tables

1. Go to VPC → Route Tables.

2. Select Muqeem-VPC-rtb-private1.

3. Click the Routes tab.

4. Click Edit routes.

5. Click Add route.

Enter:

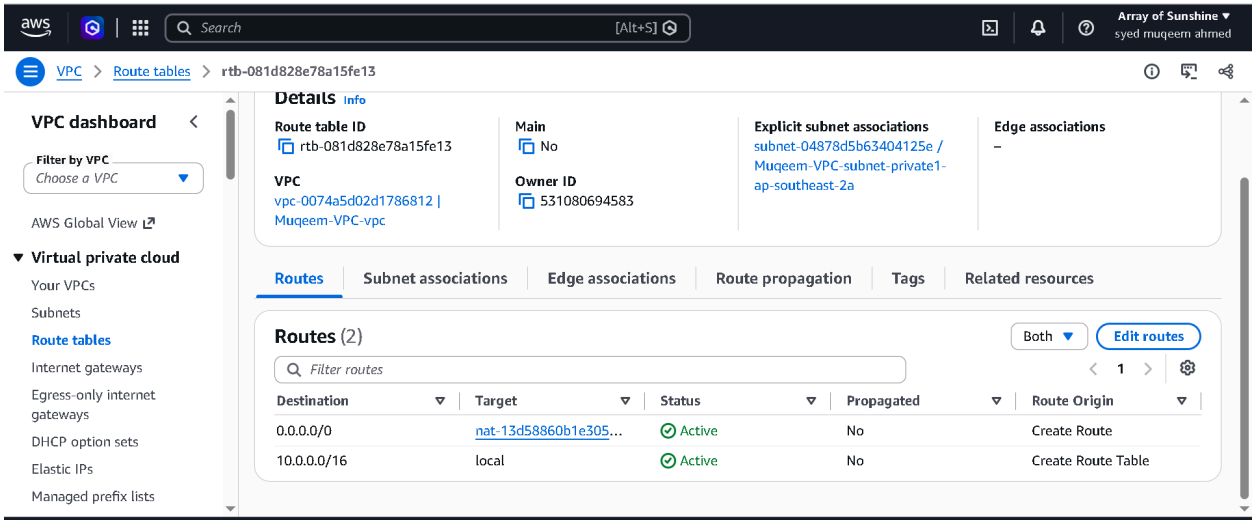
Destination	Target
0.0.0.0/0	Your NAT Gateway

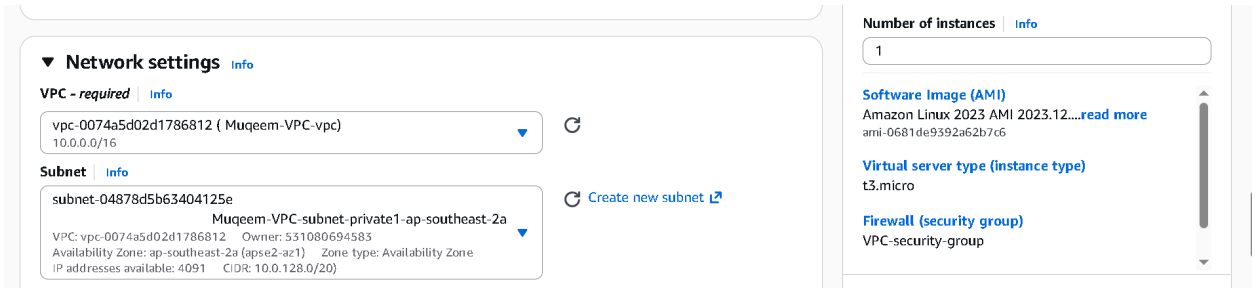
For Target:

- Select NAT Gateway.

- Choose Muqeem-NAT-Gateway.

6. Click Save changes.





#. Connect to Private EC2 using Public EC2

Step 1: Connect to Public EC2

Open your existing public EC2 instance.

Click:

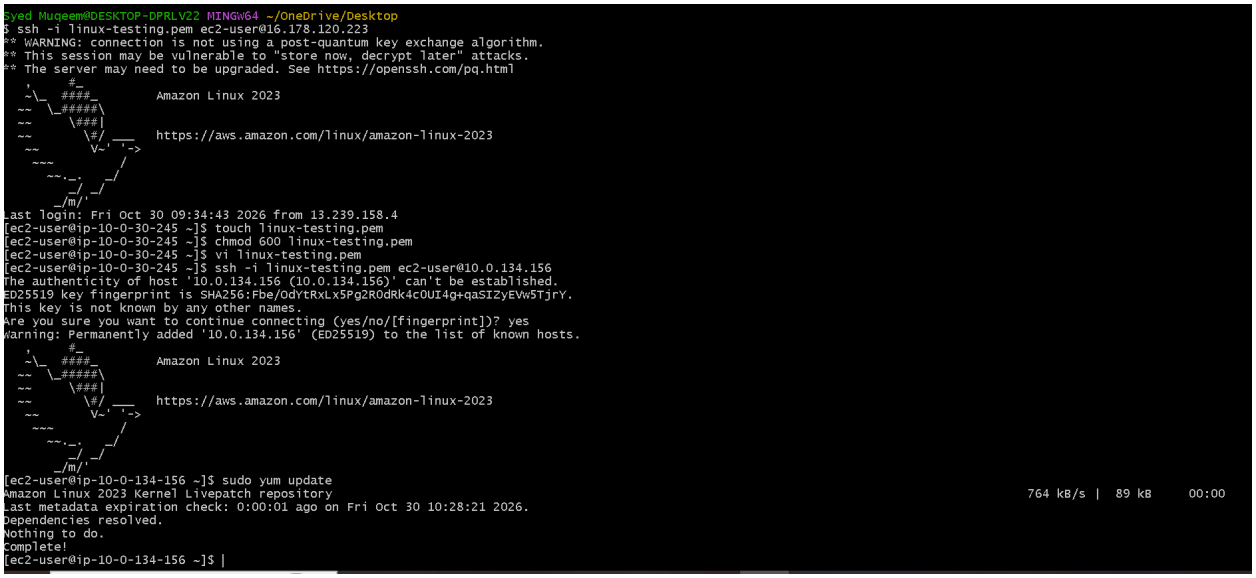
Connect → EC2 Instance Connect → Connect

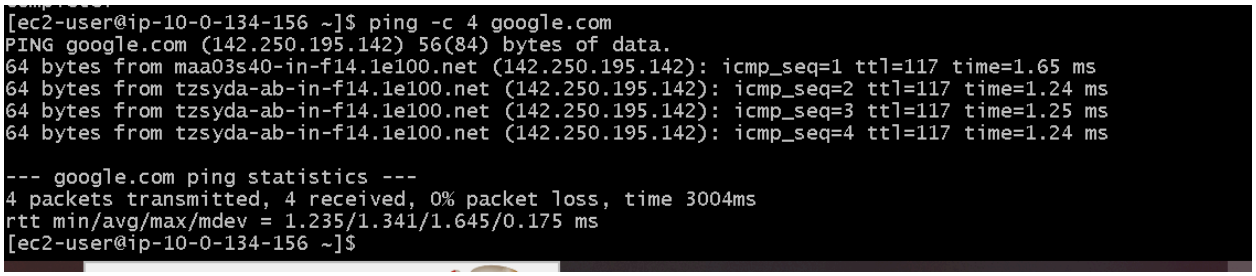
Step 2: Connect from Public EC2 to Private EC2

In the public EC2 terminal, run: ssh

6. Verify NAT Gateway connectivity

Once connected to the private EC2, run: **ping -c 4 google.com**





9. Install Apache Tomcat in private EC2 and deploy a sample app.

Install Apache Tomcat in Private EC2 and Deploy a Sample Application

1. Connected to the private EC2 instance through the Bastion Host (Public EC2).

2. Checked the operating system and confirmed that the instance was running Amazon Linux 2023.

3. Updated the system packages to ensure the instance had the latest available updates.

4. Installed Java 21 (Amazon Corretto), which is required to run Apache Tomcat.

5. Verified the Java installation and confirmed that Java 21 was installed successfully.

6. Downloaded Apache Tomcat 10.1.61 from the official Apache website.

7. Created the Tomcat installation directory under `/opt/tomcat`.

8. Extracted the downloaded Tomcat archive into the installation directory.

9. Created a dedicated Linux user named `tomcat` to run the Tomcat service securely.

10. Assigned the required ownership of the Tomcat installation directory to the Tomcat user.

11. Created a systemd service file to manage Apache Tomcat as a Linux service.

12. Configured the service with the Java and Tomcat environment variables and startup settings.

13. Reloaded the systemd configuration and enabled the Tomcat service to start automatically at system boot.

14. Started the Tomcat service and verified that its status was **Active (running)**.

15. Verified that Tomcat was listening on its default port, **8080**.

16. Downloaded the sample Java web application (WAR file).

17. Copied the sample WAR file into Tomcat's `webapps` directory for deployment.

18. Assigned the correct ownership of the sample application to the Tomcat user.

19. Restarted the Tomcat service to deploy the sample application.

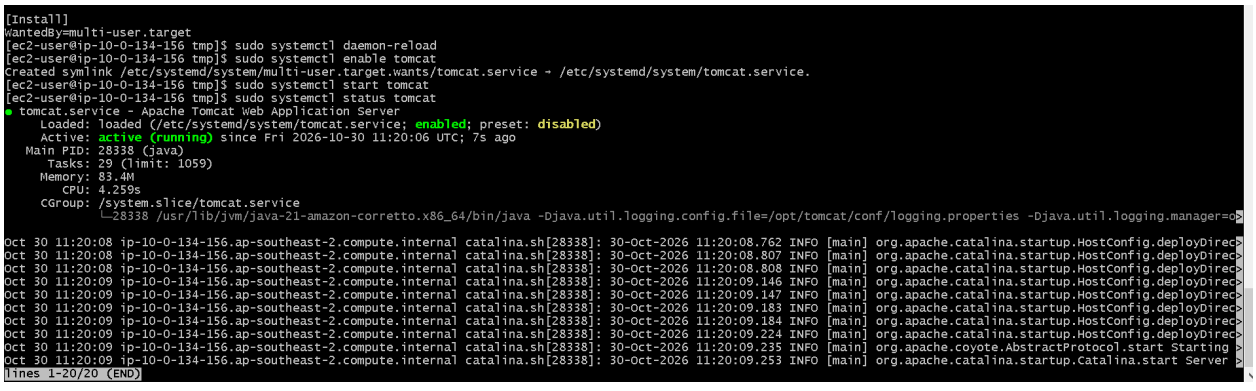
20. Verified that Tomcat successfully extracted the WAR file and created the sample application directory.

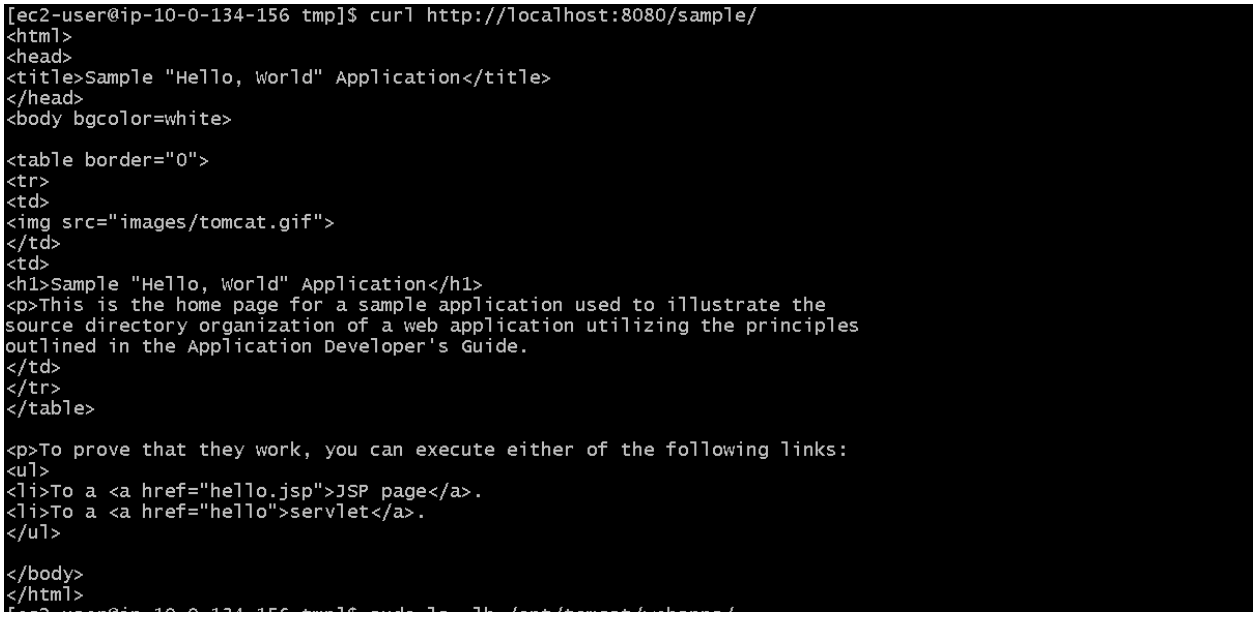
21. Tested the deployed application locally and confirmed that the Sample "Hello, World" page was displayed successfully.

22. Verified the sample JSP page and servlet endpoints.

Final Result: Apache Tomcat 10.1.61 was successfully installed and configured on the private Amazon Linux 2023 EC2 instance using Java 21. The Tomcat service was started successfully, and the sample Java web application was deployed and tested.

Note: Since the EC2 instance is in a private subnet, the application is not directly accessible from the public internet. Access can be provided securely through a Bastion Host or another private-access method.





10. Configure VPC flow logs and store the logs in S3 and CloudWatch.

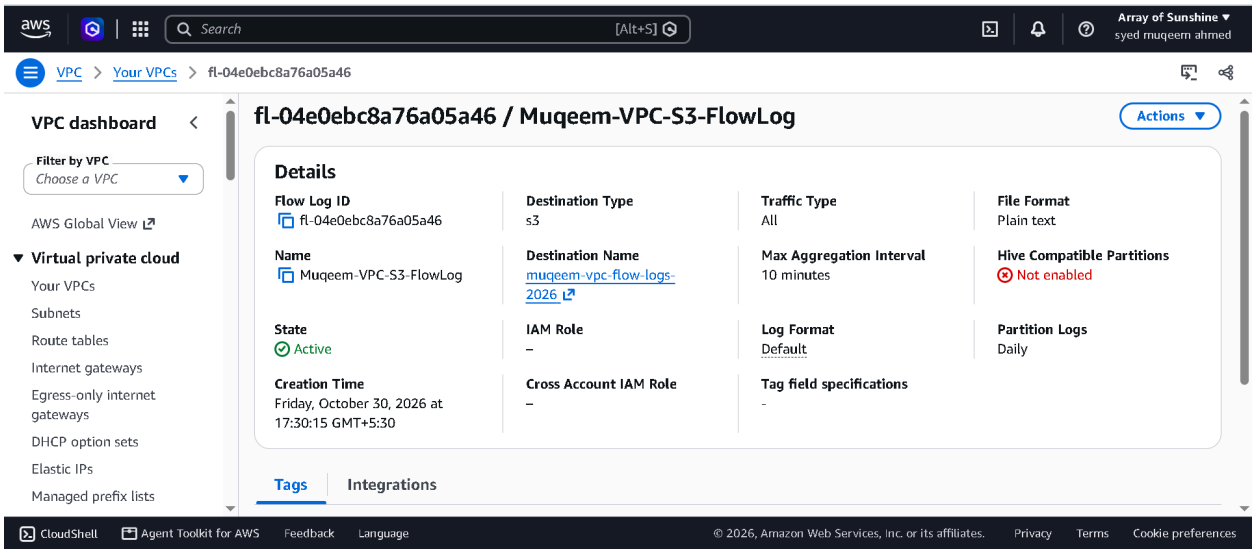
Part 1: Create an Amazon S3 Bucket

1. Open the AWS Management Console.
2. Search for and open Amazon S3.
3. Click on Create bucket.
4. Select General purpose bucket.
5. Enter the bucket name `muqeem-vpc-flow-logs-2026`.
6. Select the same AWS Region as the VPC.
7. Keep Block Public Access enabled.
8. Keep the default encryption settings enabled.
9. Click Create bucket.
10. Verify that the S3 bucket was created successfully.

Part 2: Create VPC Flow Logs for Amazon S3

11. Open the Amazon VPC console.
12. Navigate to Your VPCs.

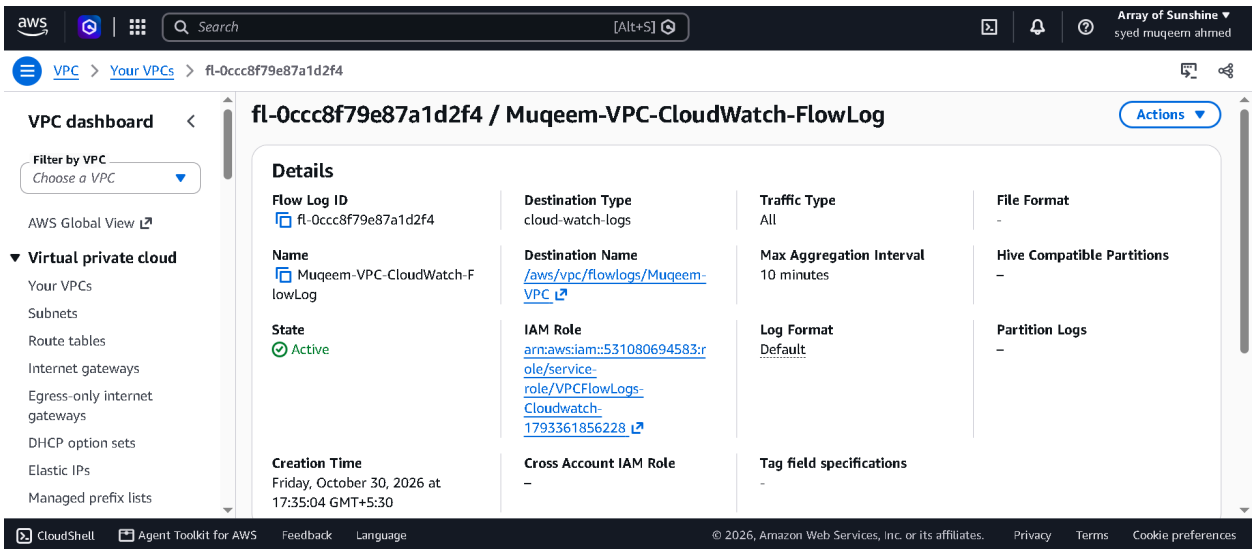
13. Select the VPC named `Muqeem-VPC-vpc`.
14. Open the Flow logs tab.
15. Click Create flow log.
16. Enter the flow log name `Muqeem-VPC-S3-FlowLog`.
17. Select All under the Filter option to capture both accepted and rejected traffic.
18. Select 10 minutes as the maximum aggregation interval.
19. Under Destination, select Send to an Amazon S3 bucket.
20. Enter the S3 bucket ARN: `arn:aws:s3:::muqeem-vpc-flow-logs-2026`.
21. Select AWS default format for the log records.
22. Keep Hive-compatible S3 prefixes disabled.
23. Keep the default daily log partitioning option.
24. Add a tag with Key `Name` and Value `Muqeem-VPC-S3-FlowLog`.
25. Click Create flow log.
26. Verify that the S3 Flow Log was created successfully and its state is Active.



Part 3: Create VPC Flow Logs for CloudWatch

27. Navigate back to Your VPCs in the VPC console.
28. Select `Muqeem-VPC-vpc`.
29. Open the Flow logs tab.
30. Click Create flow log.
31. Enter the flow log name `Muqeem-VPC-CloudWatch-FlowLog`.

32. Select All under the Filter option.
33. Select 10 minutes as the maximum aggregation interval.
34. Under Destination, select Send to CloudWatch Logs.
35. Enter the destination log group name `/aws/vpc/flowlogs/Muqeem-VPC`.
36. Select the option to create and use a new service role, if available.
37. Provide the required IAM role name, if prompted.
38. Select AWS default format for the log records.
39. Add a tag with Key `Name` and Value `Muqeem-VPC-CloudWatch-FlowLog`.
40. Click Create flow log.
41. Verify that the CloudWatch Flow Log was created successfully and its state is Active.



Part 4: Verify Logs in CloudWatch

42. Open Amazon CloudWatch from the AWS Management Console.
43. Navigate to Logs and select Log groups.
44. Open `/aws/vpc/flowlogs/Muqeem-VPC`.
45. Check whether log streams have been created.
46. Open a log stream to inspect the VPC traffic records.

Part 5: Verify Logs in Amazon S3

47. Open Amazon S3 from the AWS Management Console.

48. Open the bucket `muqeem-vpc-flow-logs-2026`.
49. Locate the AWS-generated `AWSLogs` folder.
50. Open the folder structure and check for the delivered VPC Flow Log files.
51. Verify that the log files contain network traffic records.

Final Result

Successfully configured VPC Flow Logs for the `Muqeem-VPC` and created two separate Flow Logs.

- Amazon S3 Flow Log: `Muqeem-VPC-S3-FlowLog`

- CloudWatch Flow Log: `Muqeem-VPC-CloudWatch-FlowLog`

VPC Flow Logs were configured to capture all accepted and rejected network traffic and deliver the records to both Amazon S3 and Amazon CloudWatch Logs.

Flow logs Status:

